Exercise 2: Introduction to Arduino

Introduction

In this exercise, you will build and test a multi-element circuit forming a functional alarm clock. Using the Arduino Uno, an LCD screen, a buzzer, a real-time clock, and few control elements, you will build a programmable system that keeps the time and rings an alarm when set.

Record your work as you go — take photos of test circuits configuration and note down all measurements and observations. You will need this material when writing up your portfolio report. Record a video of a functioning alarm clock once you build it.

All components and instruments can be found in the exercise box.
This instruction covers the necessary elements one by one. In addition to those, you may need the E*lectronic Box* toolbox throughout the entire exercise – please ask your lab tutor to provide it when needed.

Important: This instruction does not provide step by step guidance to build the system! It explains individual elements you can use to build your own project.

Since our course is primarily focused on electronics, we provide you an example code for a working alarm clock and smaller snippets for testing components. However, we strongly encourage you to come up with your own code, both for test circuits and the alarm clock itself. Please beat us and make your clock even cooler!

All necesarry files – libraries, test codes, working example, etc. can be found in the course page in Stud.IP.

Few General Rules

- By default, **power the circuit through the USB socket from your computer** or power supply**,** unless specified otherwise in the manual. **When powering via USB, you can ignore the source/ground connected to V_{in} and GND pins of the Arduino in the schematics** – USB powers and grounds Arduino for you.

- Remember to **share common ground** using the GND pin on the Arduino with other components.

- **Always switch off and disconnect power before modifying or rewiring a circuit.** Never change connections while the circuit is live.

- **If anything smells, sparks, or gets unexpectedly hot — immediately disconnect power and inform the tutor before doing anything else.**
- **Double-check your wiring against the schematic before applying power** for the first time. Pay particular attention to polarity — connecting a component backwards can damage it permanently.
- Ensure that **all your circuits have one common ground.**
- **Mind the rated voltages for particular components** - some may work with 5V logic, some with 3.3V. Arduino provides output for both, use them appropriately.
- Use Google and consult data sheets and manuals for particular components, if you're unsure about their operation. Arduino prototyping is rooted in community knowledge sharing.

Preparing your environment

If you have not been using Arduino environment before, you need to start with downloading the Arduino IDE client. This program will be your headquarters for all things Arduino[1].

You can download the Arduino IDE for your system:
https://docs.arduino.cc/software/ide/

Additionally, we will need some extra libraries to make using our components easier. Please visit the Stud.IP course page and download the package prepared for this exercise.
The package contains the necessary libraries to install and example codes we will be using during the class.

Sub-circuit 1 – Connecting the buzzer

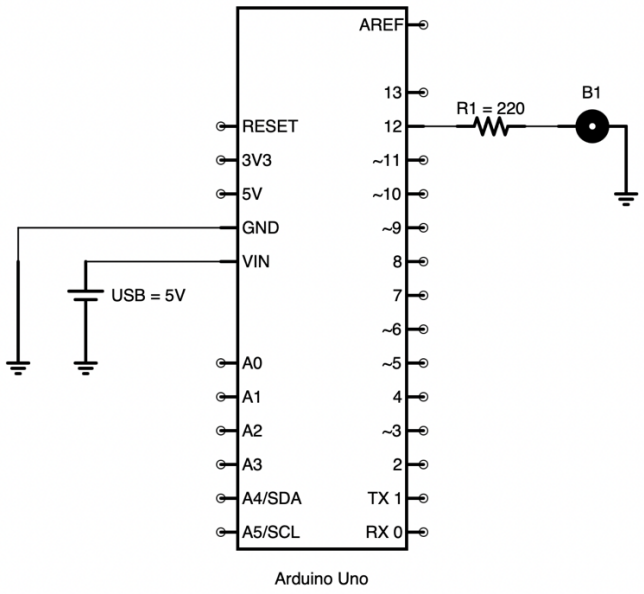
In this task, you will start by building a simple circuit to light up an LED. Through the subtasks, you will consecutively add more control over the circuit, introducing a switch and then a potentiometer to adjust the brightness.
Components required:

- Arduino Uno
- Buzzer
- Resistor

[1] Arduino can be programmed in other environments too – Visual Studio, Sublime, Notepad++, etc. with appropriate plugins. However, if you use them, we will not be able to assist you if you run into any environment config problems. We suggest using your favorite alternative only if you're a proficient user.

- Breadboard
- Set of male-male jumper wires

Compose the circuit as shown in the schematic:



Run the test code from the file buzzer_test.ino and consult the comments in the code. Experiment with changing the test conditions – duration of delays, sequence of HIGH/LOW states.

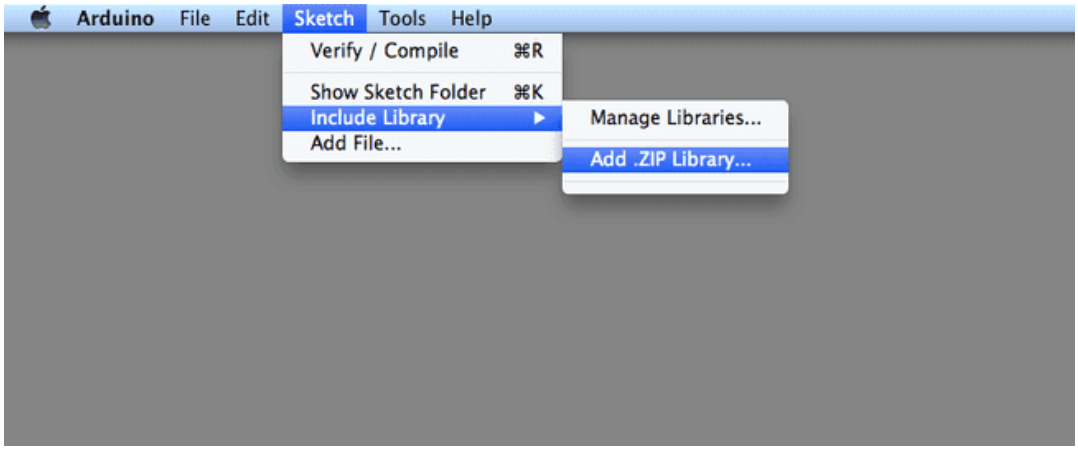
What have you observed?

Remember to photograph or record short videos of your circuits in operation.

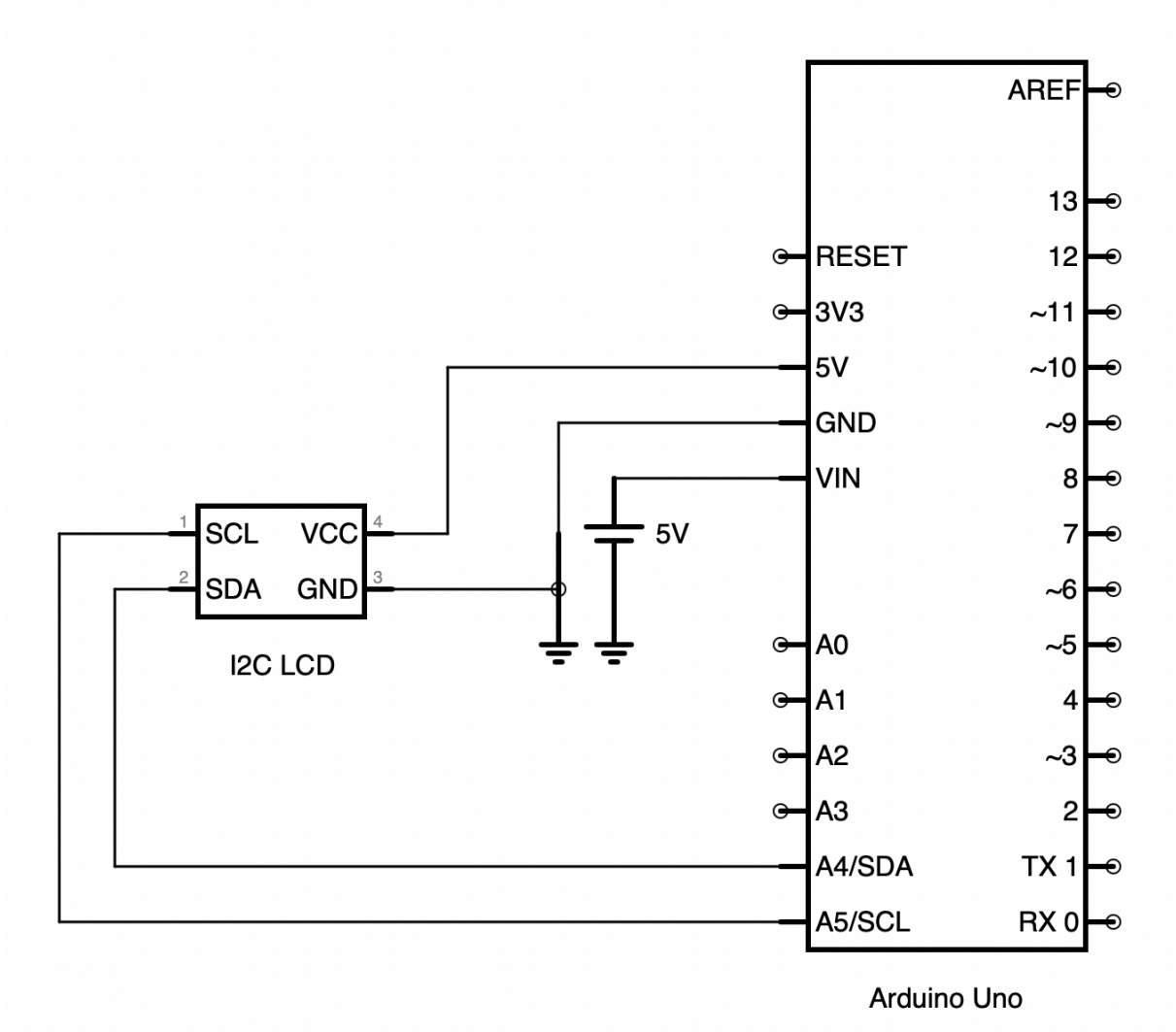
Sub-circuit 2 – Connecting the LCD screen

The provided LCD is 2x16 sign display communicating over I2C protocol. The I2C addressing allows us to communicate with the display using only 4 wires.

To do this effectively, we will need to include the library for our screen. In Stud.IP you can find the .zip file **with the library <LiquidCrystal_I2C.h>.** Install the library and include it in your .ino file.



Compose the circuit as shown in the schematic:



I²C uses two wires: **SCL** is like a clock that sets the timing, and **SDA** is the wire that carries the actual data. The master device controls the clock and tells other devices

when to send or receive bits. All devices share the same wires, taking turns to communicate without talking over each other.

In I²C, each device has its own address, so the master knows who it wants to talk to. The master first sends this address on the SDA line, and every device listens, but only the one with the matching address responds. After that, the master and that specific device can send data back and forth while the others stay quiet.

Therefore, to be able to communicate with the LCD, we will need to determine the address of the LCD screen. Typical values are 0x20 or 0x27, but it is not guaranteed.

To this end, use the I2C_scanner.ino sketch. Run it on your Arduino and observe the serial port output. If you connected the screen correctly, the scanner should tell you the address of your LCD.

Then, proceed with LCD_test.ino. This sketch provides you with a simple script that prints out text on the LCD. Analyse the contents of this file, together with the comments, to learn how to operate the LCD.

Sub-circuit 3 – Expanding the setup with a Real Time Clock

At this stage, we can expand out circuit with another device using I²C communication – the Real Time Clock (RTC) module. This chip enables your system to depend on the real time kept and processed by the RTC. You can notice that the RTC module requires a cell battery – thanks to this power source, parts of the module work independently and can keep the time even when you turn off the circuit.

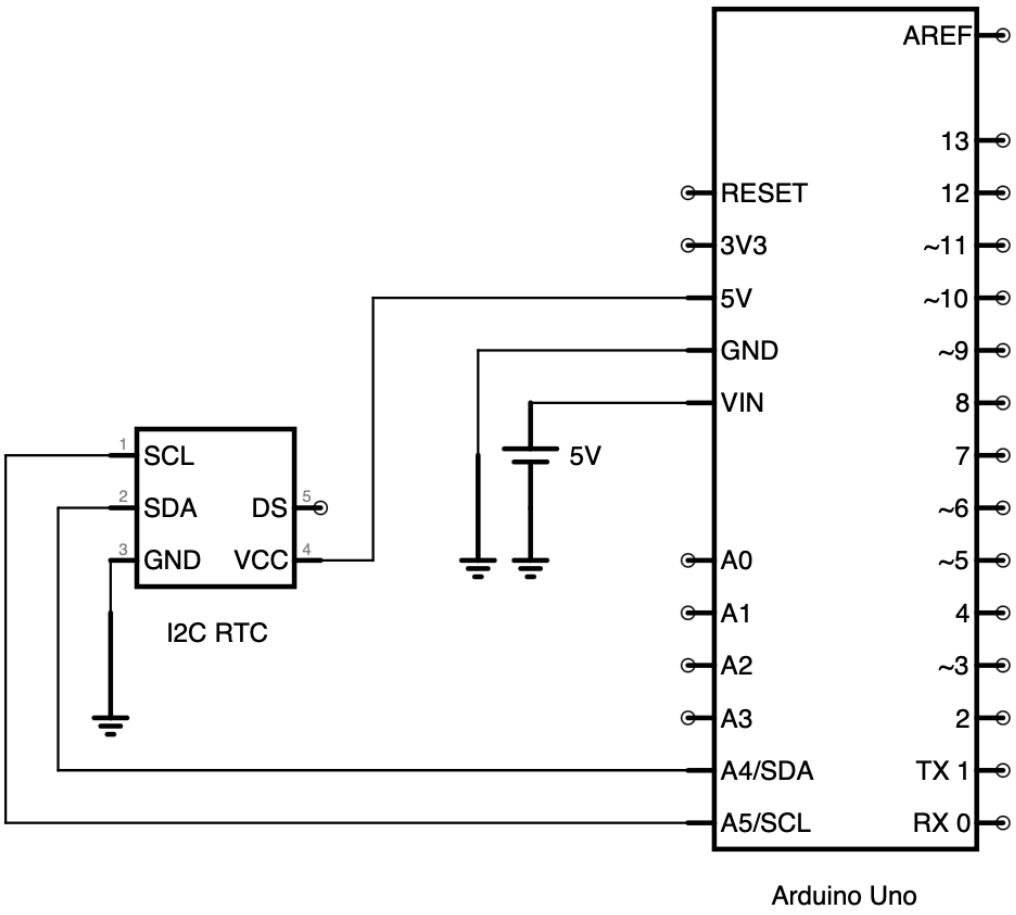
To operate the RTC module, you'll need the **RTClib.h library.**

The RTC module is using I²C communication, therefore we need to know its address to configure it. To this end, use the I2C_scanner.ino sketch. Run it on your Arduino and observe the serial port output. If you connected the RTC correctly, the scanner should tell you the address of your RTC. When connected parallel to your LCD, two addresses will be reported. If you completed the previous steps, you already know your LCD address and can easily differentiate which device has which address.

Then, proceed with RTC_LCD_test.ino. This sketch provides you with a simple script that sets up and processes the readings of the RTC. Analyse the contents of this file, together with the comments, to learn how to operate the RTC module.

On the next page, you'll find the schematic on how to connect the RTC module.

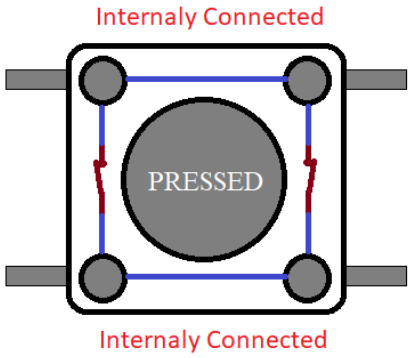
Compose the circuit as shown in the schematic:



Sub-circuit 4 – Using the Push Button

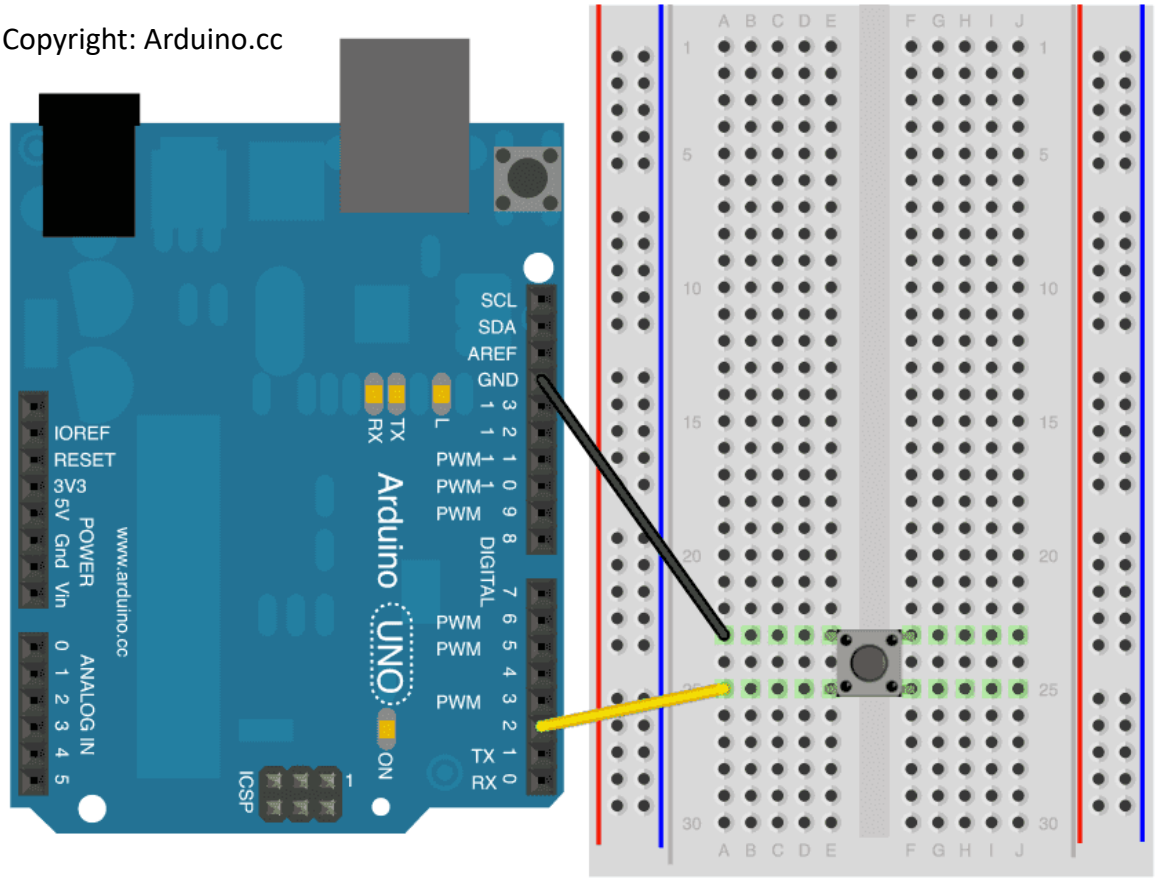
At this stage, we can expand out circuit with a button interface. To this end, we will use instantaneous push buttons.

Please mind that push buttons have two pairs of legs; which are pairwise connected. The connection with the opposite pair is made when you push the button.



You can easily check which pair is which using the multimeter in continuity check mode.

The easiest way to connect the button to Arduino is to use the in-build pull-up resistor.



To make such connection work, it is important to include in your **setup() function** a declaration that this digital pin will work with pull-up resistor:
pinMode(2, INPUT_PULLUP);

Having connected the button, you can read its value using regular digitalRead().

If you're using the Button.h library, the step above is done by the library when you initialise the button.

You can preview a working direct example here:
https://docs.arduino.cc/tutorials/generic/digital-input-pullup/

To make operation of the buttons easier, we recommend using **Button.h library.**
This library handles debouncing and other annoying details for you.
You can find it here, together with example codes:
https://github.com/madleech/Button

The library zip file is provided in the exercise file package.

Your Task – Build an Alarm Clock

You now know how to operate various devices – a display, buzzer, real time clock, buttons. The set also offers you few more components. Using those, we invite you to build a functional circuit on your own and program it.

Your task is to create a functional alarm clock – one that will keep the current time, and allow for setting up the alarm time, rang the noise when the set time strikes, and allow for turning the alarm off – all without having to make changes in the code.
We strongly encourage you to take this idea further – maybe a snooze function? Other controls then just push buttons? The kit offers a few options, but we're open for your requests of additional components.

The basic version of this exercise that allows to obtain a passing grade is building a circuit that will work with the example code provided – see file DDF_Arduino101_AlarmClock.ino

However, wiring it up so it runs the provided code is the minimal version (and only awards a grade of 3.0) if it does not contain substantial improvements.

We strongly encourage you to ideate the circuitry and code from scratch and make this fully your design. We're super happy to see you beat our design and make awesome alarm clocks!